DATA 377 PS 2

Ananth Malladi

November 2026

1 Problem 1

1.1 Part a

Let the outputs of the network be functions g_1, g_2, where $g_1 : \mathbb{R} \to \mathbb{R}$, $g_2 : \mathbb{R} \to \mathbb{R}$ For a general real-valued random variable Y, we can set $f_\mu = g_1$ as our prediction for the mean $\mathbb{E}[Y|X]$. Because the standard deviation is positive, it is reasonable to choose $f_\sigma(x) = \exp(g_2(x))$. However, since the exponential grows quickly (and can therefore lead to numerical overflows), it is more stable to use the softplus function, $f_\sigma(x) = \log(1 + e^x)$. I'm using the exponential for my function here, though.

1.2 Part b

Modeling our data by a normal distribution,

$$p(y|x) = \frac{1}{\sqrt{2\pi f_\sigma(x)^2}} e^{-\frac{(y - f_\mu(x))^2}{2 f_\sigma(x)^2}}$$

we can compute the loss function by maximizing the likelihood of the observations y. This is equivalent to minimizing the negative log likelihood, which is

$$L(y, f(x)) = -\log p(y|x) = \text{const.} + \log f_\sigma(x) + \frac{(y - f_\mu(x))^2}{2 f_\sigma(x)^2}$$

1.3 Part c

I do this in the code, comparing against three baselines: a linear model, an SVM, and sklearn's MLP (see the code for the hyperparameters of each of these models).

Baseline	Val MSE	σ (from training residuals)	Val loss
Linear	0.9483	0.9720	0.4735
SVR	0.3598	0.5324	0.0043
sklearn MLP	0.2938	0.5363	-0.1124

```
Validation error for model with hidden layers [7]:0.08724603801965714
Validation error for model with hidden layers [5]:0.13904766738414764
Validation error for model with hidden layers [9]:-0.15524375438690186
Validation error for model with hidden layers [7, 7]:0.06961434334516525
Validation error for model with hidden layers [7, 7, 7]:0.019539330154657364
```

Figure 1: Enter Caption

The sklearn MLP is the best baseline model, and is only beaten by my MLP with a single hidden layer of dimension 9. This is the model I choose as my optimal model.

1.4 Part d

Given input data x, the model outputs f_μ, f_σ, from which our estimate of the probability distribution is

$$p(y|x) = \frac{1}{\sqrt{2\pi f_\sigma(x)^2}} e^{-\frac{(y-f_\mu(x))^2}{2f_\sigma(x)^2}}$$

From here, we'd predict the probability that a trip takes less than t minutes as:

$$p(y < t|x) = \int_{-\infty}^{t} dy \; \frac{1}{\sqrt{2\pi f_\sigma(x)^2}} e^{-\frac{(y-f_\mu(x))^2}{2f_\sigma(x)^2}}$$

1.5 Part e

Performing the procedure described in the problem set, I find a cross-entropy loss of 0.019, and I find that 99.6% of my predictions are accurate for my best model.

2 Problem 2

2.1 Part a

One hot encoding assigns a different unit basis vector to every input. In this case, there are 3665 stock codes and 4339 customer ID's, which means that under one-hot encoding, the dimensionality of the input space is 8004. This is too large for one-hot encoding to be feasible.

2.2 Part b

I'll use the (negative log likelihood of the) negative binomial distribution, a heavy-tailed distribution over the natural integers, to define the loss function. I use one hidden layer with dimension 128. I compare it against three baselines: one using a Poisson distribution (with the same hidden layer structure), one negative binomial with no hidden layers, and one Poisson with no hidden layers.

I use early stopping on all of them. Using target encoding, I find the following loss curves for the three baseline models:

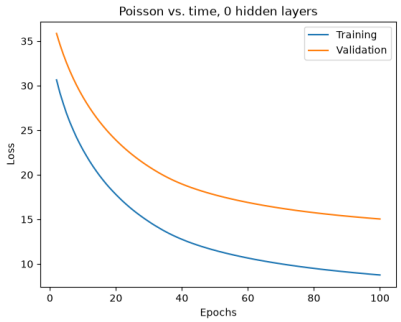
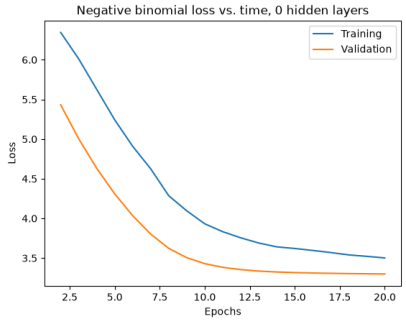
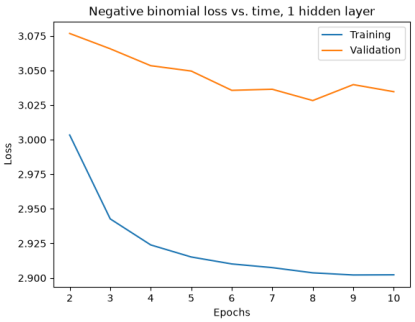


Figure 2: Enter Caption



And I find the following for my proposed negative binomial + 1 hidden layer model:



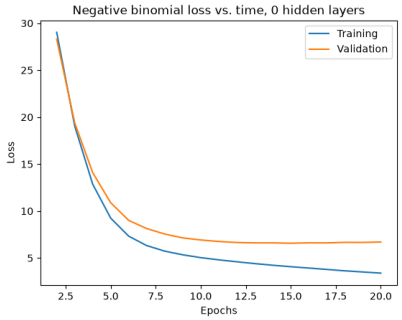
Here are the val/train/test losses of the various models.

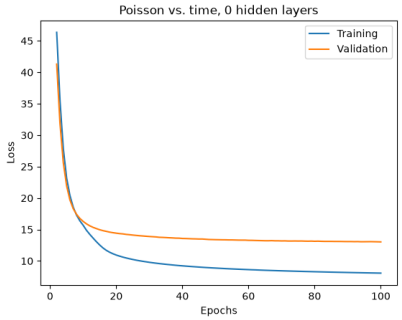
Model	Train	Val	Test
NB, 1 hidden layer	2.8978	3.0347	2.9380
Poisson, 1 hidden layer	8.2231	13.0675	6.4585
NB, 0 hidden layers	3.4849	3.2995	3.2233
Poisson, 0 hidden layers	8.7418	15.0412	8.1448

As expected the negative binomial 1 hidden layer model performs the best on the validation set, which is therefore what I should pick here.

2.3 Part c

Using dense encoding into $\mathbb{R}^d, d = 30$, I repeat essentially the same procedure as part b. I find the following loss curves for my baseline models:





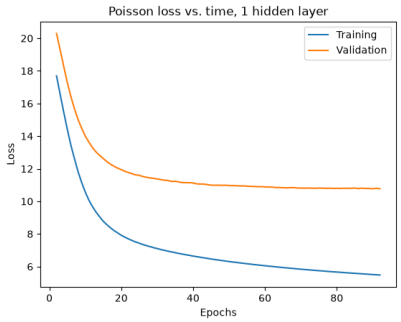
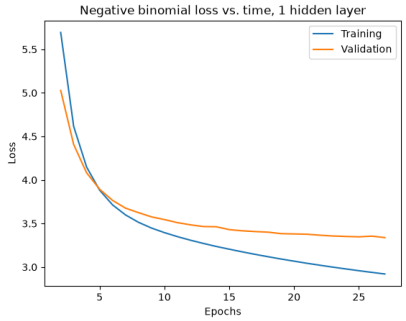


Figure 3: Enter Caption

And I find the following for my proposed negative binomial + 1 hidden layer model:



Here are the val/train/test losses

Model	Train	Val	Test
NB, 1 hidden	2.9053	3.3385	3.1377
Poisson, 1 hidden	5.4285	10.7698	5.6036
NB, 0 hidden	3.2785	6.7044	2.9492
Poisson, 0 hidden	8.0108	13.0283	6.2837

Figure 4: Enter Caption

Seems like the negative binomial models both perform quite well, with the 1 hidden layer one performing best on the validation set.

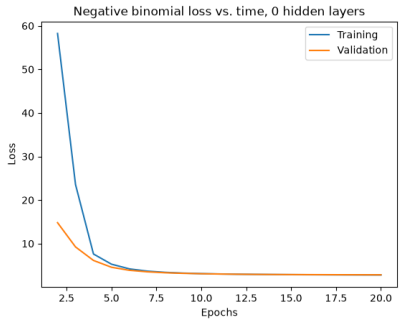


Figure 5: Enter Caption

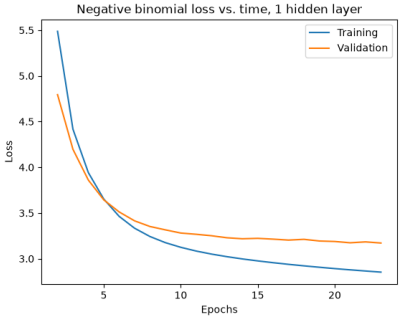


Figure 6: Enter Caption

2.4 Part d

We do the same thing.

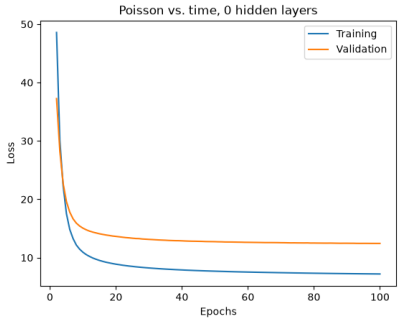


Figure 7: Enter Caption

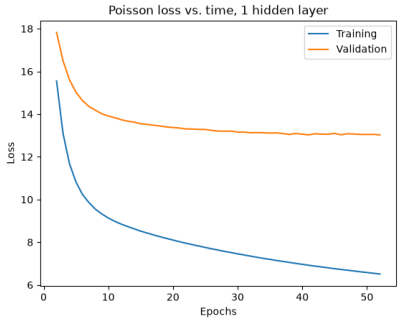


Figure 8: Enter Caption

Model	Train	Val	Test
NB, 1 hidden	2.8408	3.1714	3.0321
Poisson, 1 hidden	6.4702	13.0253	6.0002
NB, 0 hidden	2.7750	2.8292	2.8325
Poisson, 0 hidden	7.1559	12.4351	6.2264

Surprisingly, we find that the 0 hidden layer negative binomial model appears to perform the best on the validation set. That would be the one we choose, then.

2.5 Part e

Intuitively, adding an embedding layer should improve performance, since it the model is able to optimize how efficiently it encodes data (as opposed to a target encoder, for example). Target encoding is a bit weird, since we're replacing something alphanumeric (like stock codes) with a number, and indeed those models don't perform as well in general. Target encoding is also complicated by the fact that there a few very large outliers, which will shift the means significantly. Doing both gives the model more things to work with, and as expected that performs the best.

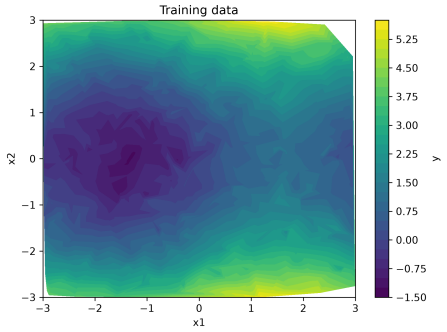
The goal of the embedding layer is to put vectors that mean similar things close to each-other. It's hard to get a sense of what stock codes mean by just staring at them, but clearly they have meaning when thought of in terms of their effect on the quantity. That's why the embedding layer is useful in this context – it essentially learns what these mean and how to best represent them in a way that the network can handle.

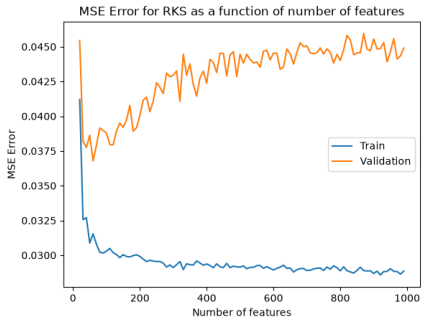
Based on the testing loss, it seems like the model does not need hidden layers if the data is encoded well enough. This is a bit surprising – I think this is mostly a peculiarity of my random seeding, and should go away on average. i.e. I expect that the NB 1 hidden layer model should perform best on average.

3 Problem 3

3.1 Part a

Here's what the training data looks like.



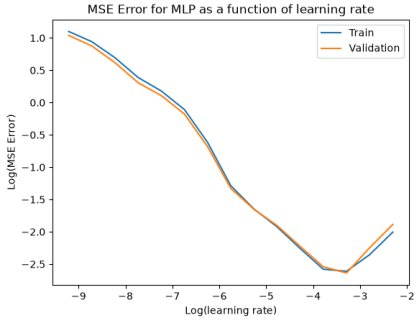


3.2 Part b

Fixing the standard deviation of the normal distribution from which the coefficients w are chosen to be 1, and fixing the ridge regression regularization strength to be $\lambda = 10^{-3}$, we sweep over the number of functions D. We plot this sweep below:

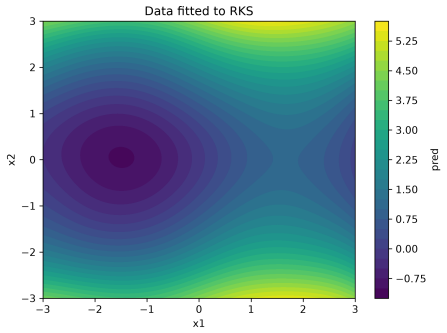
3.3 Part c

We perform a sweep over learning rate, with a fixed single hidden layer with dimension 128. This is what the sweep over learning rate looks like:

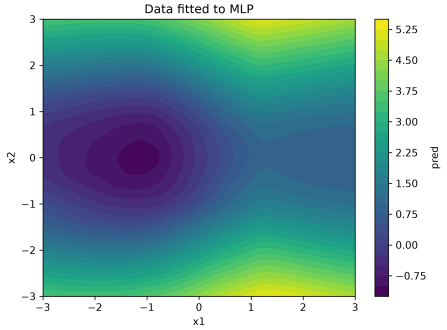


3.4 Part d

Here is what the random kitchen sink fit looks like



This is what the MLP fit looks like



Comparing to the data shown in part a, both models qualitatively capture the structure of the data. It appears that the MLP's fit is a bit less symmetrical, but it is a bit worse from the standpoint of mean squared error. Both models capture the blue troughs, and give peaks at $x_2 = \pm 3, x_1 \sim 1$.

See the files on the GitHub for comparisons to other models – the linear model doesn't really capture the right features qualitatively, and a regression tree is able to capture the overall structure but has a higher loss than the MLP.

It appears that the RKS model performs the best here, which is what I would choose based on the validation loss.

I give the losses/parameter values for the various models here.

```
Best D for RKS:  60
Best validation MSE of RKS:  0.036782695504155596
Best learning rate for MLP:  0.03727593720314938
Best validation MSE of MLP:  0.07152802789496626
Validation MSE of linear regression:  2.0592785551578072
Best max_depth for regression tree:  10
Best validation MSE of regression tree:  0.08880654319276417
```